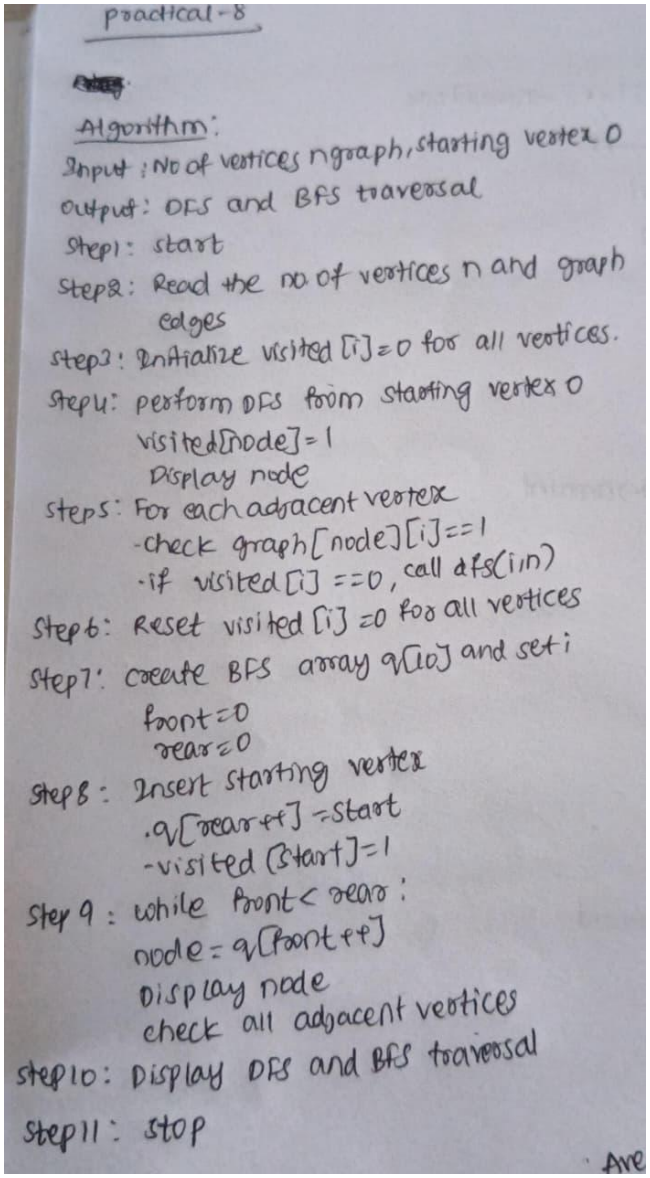
PRACTICAL-8

AIM: To Implementation of Graph and Searching (DFS and BFS).

Graph and Searching (DFS and BFS)

Algorithm:



PROGRAM:

#include <iostream>

```
using namespace std;

class Graph {
    int V;
    int adj[100][100];

public:
    Graph(int vertices) {
        V = vertices;

        for (int i = 0; i < V; i++)
            for (int j = 0; j < V; j++)
                adj[i][j] = 0;
    }

    void addEdge(int u, int v) {
        adj[u][v] = 1;
        adj[v][u] = 1;
    }

    // DFS
    void DFSUtil(int v, int visited[]) {
        visited[v] = 1;
        cout << v << " ";

        for (int i = 0; i < V; i++) {
```

```
        if (adj[v][i] == 1 && visited[i] == 0) {
            DFSUtil(i, visited);
        }
    }
}

void DFS(int start) {
    int visited[100] = {0};
    DFSUtil(start, visited);
}

// BFS
void BFS(int start) {
    int visited[100] = {0};
    int q[100];

    int front = 0, rear = 0;

    visited[start] = 1;
    q[rear++] = start;

    while (front < rear) {
        int node = q[front++];
        cout << node << " ";

        for (int i = 0; i < V; i++) {
```

```
            if (adj[node][i] == 1 && visited[i] == 0) {
                visited[i] = 1;
                q[rear++] = i;
            }
        }
    }
}
};

int main() {
    int V, E;

    cout << "Enter number of vertices: ";
    cin >> V;

    Graph g(V);

    cout << "Enter number of edges: ";
    cin >> E;

    cout << "Enter edges (u v):\n";

    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v);
```

```
    }

    int start;

    cout << "Enter starting vertex: ";
    cin >> start;

    cout << "\nDFS Traversal: ";
    g.DFS(start);

    cout << "\nBFS Traversal: ";
    g.BFS(start);

    return 0;
}
```

OUTPUT:

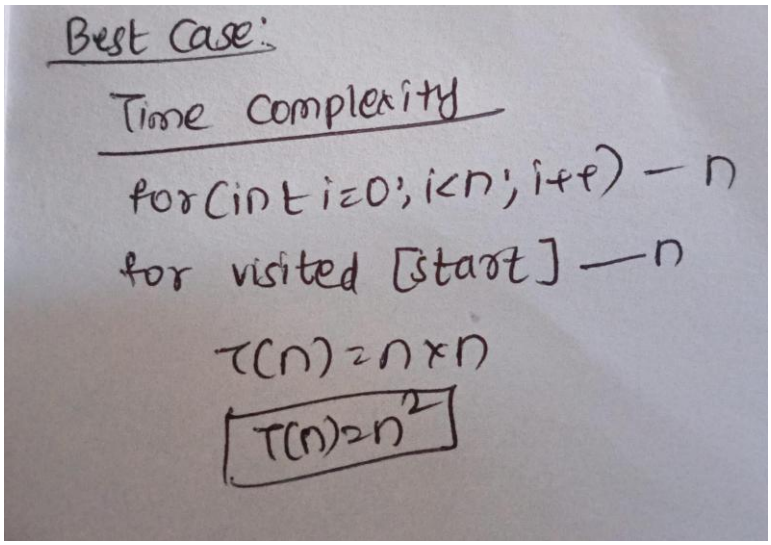
```
Output

Enter number of vertices: 4
Enter number of edges: 4
Enter edges (u v):
0 1
2 1
0 1
0 7
Enter starting vertex: 1

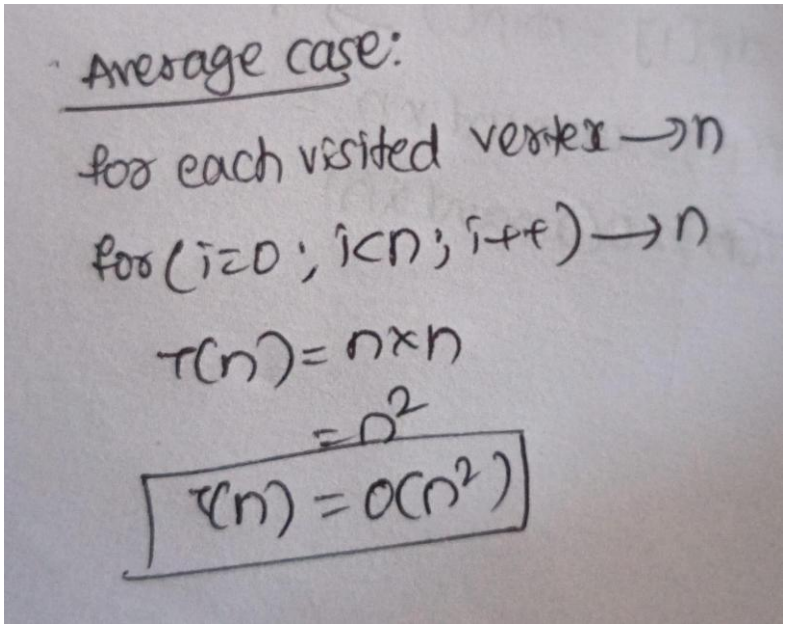
DFS Traversal: 1 0 2
BFS Traversal: 1 0 2
```

Time Complexity:

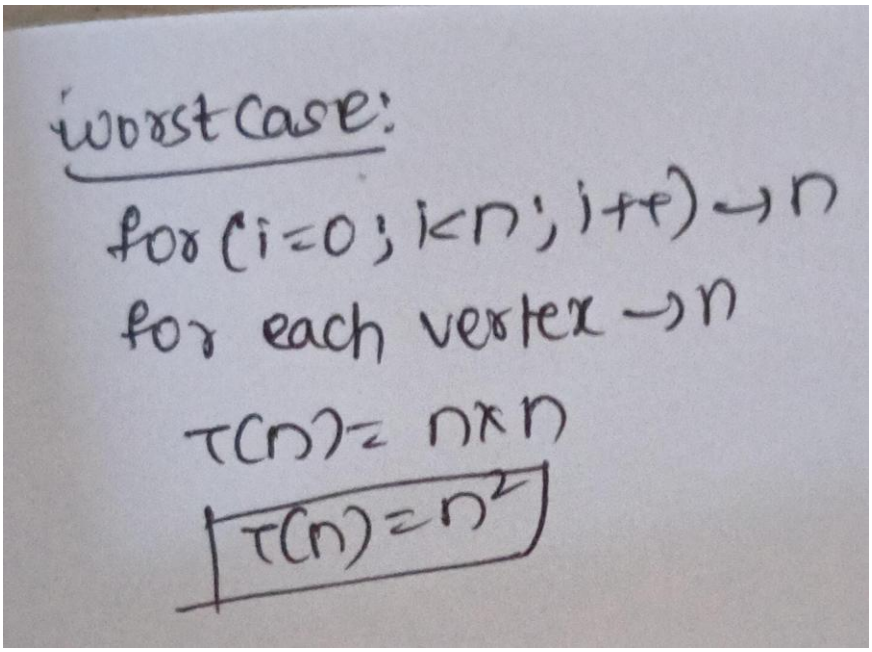
Best Case:



Average Case:



Worst Case:



Conclusion:

The implementation of BFS and DFS traversal helps us understand graph traversal techniques. DFS visits vertices using recursion, while BFS visits vertices level by level using an array. Both algorithms provide systematic ways to traverse a graph. For the given adjacency matrix implementation, both BFS and DFS have a time complexity of $O(n^2)$ in the best, average, and worst cases. Therefore, BFS and DFS are useful and efficient techniques for traversing graphs.